

Optimizacion de costos - Matriz de decision de modelos

Optimización de costos — Matriz de decisión de modelos

Glowing Beauty S.R.L. — Automatización de recepción de facturas Departamento de Compras Septiembre 2026

1. Resumen ejecutivo

El flujo no usa un modelo: usa **el modelo mínimo suficiente para cada tarea**, y en varias tareas no usa ninguno.

Decisión	Resultado
Modelo principal (extracción)	Gemini 3.1 Flash-Lite
Modelo de reserva (reintento)	Claude Haiku 4.5
Redacción de correos y avisos	Plantillas, sin IA
Costo mensual proyectado	USD 0,22 sobre 400 facturas
Costo si se usara un modelo de frontera	USD 4,40 — 20 veces más

El fundamento de todo el documento es que **la IA extrae, no calcula ni decide**. Las cuentas las hace un nodo de código y las aprobaciones un humano. Eso acota el daño de un error de lectura y permite elegir el modelo barato sin asumir riesgo real.

2. Las tareas del flujo

Antes de elegir modelos hay que separar qué hace cada parte del proceso. No todas necesitan IA.

#	Tarea	Naturaleza	¿Necesita IA?
1	Validaciones de sobre (asunto, CUIT contra legajo, adjunto, tamaño)	Reglas determinísticas	No
2	Extracción de datos de la factura	Lectura estructurada de texto corto	Sí
3	Verificación aritmética (neto + IVA = total)	Cálculo	No
4	Cálculo de fecha de vencimiento	Cálculo de fechas	No
5	Reintento cuando la confianza es baja	Lectura estructurada, segunda opinión	Sí
6	Mensaje de aprobación por Telegram	Texto fijo con variables	No
7	Correo de rechazo al proveedor	Texto fijo con variables	No
8	Correo de confirmación de pago	Texto fijo con variables	No

De ocho tareas, dos usan IA. Las seis restantes se resuelven con código o plantillas. Esta es la primera y más grande optimización de costos del sistema: el modelo más barato es el que no se llama.

Las tareas 6, 7 y 8 son candidatas obvias a "que la IA redacte el mail", y se descartaron a propósito. Un correo de rechazo tiene que decir siempre lo mismo ante el mismo código: es un documento operativo con valor contractual, no un texto que convenga variar. Una plantilla es más barata, más rápida y auditable.

3. Supuestos de volumen y consumo

Parámetro	Valor	Origen
Facturas recibidas por mes	400	Volumen estimado del departamento de Compras
Tokens de entrada por factura	~1.000	Prompt de sistema (~500) + texto de la factura (~500)
Tokens de salida por factura	~200	JSON de respuesta
Entrada mensual	0,40 M tokens	400 × 1.000
Salida mensual	0,08 M tokens	400 × 200

El texto de una factura, ya extraído del PDF, ronda los 1.800 caracteres. La ventana de contexto es irrelevante acá: ningún modelo evaluado queda cerca de su límite, así que no es un criterio de decisión.

4. Precios de lista

Precios por millón de tokens, en USD, tarifa estándar sin batch ni caché.

Modelo	Entrada	Salida	Categoría
GPT-5.6 Luna	0,20	1,20	Económico
Gemini 3.1 Flash-Lite	0,25	1,50	Económico
Claude Haiku 4.5	1,00	5,00	Económico-medio
Claude Sonnet 5	3,00	15,00	Medio
GPT-5.6 Sol	5,00	30,00	Frontera

5. Costo mensual proyectado

Aplicado a la tarea 2, la extracción, que es la que concentra el volumen.

Modelo	Entrada	Salida	Total mensual	Por factura
GPT-5.6 Luna	0,080	0,096	0,18	0,00044
Gemini 3.1 Flash-Lite	0,100	0,120	0,22	0,00055
Claude Haiku 4.5	0,400	0,400	0,80	0,00200
Claude Sonnet 5	1,200	1,200	2,40	0,00600
GPT-5.6 Sol	2,000	2,400	4,40	0,01100

La diferencia entre el más barato y el de frontera es de **24 veces**. En términos absolutos son centavos por mes, pero el ejercicio no es ahorrar dos dólares: es demostrar que el modelo se elige por adecuación a la tarea. A 40.000 facturas mensuales, la misma decisión son 18 dólares contra 440.

6. Criterios de evaluación

Criterio	Peso	Por qué
Fidelidad numérica	35%	Un dígito mal leído es el peor error posible.
Adherencia a JSON	30%	Si la respuesta no parsea, el flujo se corta. Los modelos con salida estructurada por esquema eliminan esta clase de error.
Costo por factura	25%	Ver tabla anterior.

Latencia	10%	El proceso es asíncrono: nadie espera la respuesta en pantalla. Poco relevante.
----------	-----	---

La capacidad de razonamiento no se pondera: la tarea no la usa, porque los cálculos están fuera del modelo.

7. Análisis comparativo

Los cinco modelos evaluados contra los criterios anteriores, aplicados a la tarea de extracción.

Modelo	Fidelidad	JSON válido	Costo/factura	Veredicto
Gemini 3.1 Flash-Lite	Alta	Garantizado por esquema	0,00055	Elegido
GPT-5.6 Luna	Alta	Sin esquema estricto	0,00044	Descartado
Claude Haiku 4.5	Alta	Confiable	0,00200	Reserva
Claude Sonnet 5	Alta	Confiable	0,00600	Sobredimensionado
GPT-5.6 Sol	Alta	Confiable	0,01100	Sobredimensionado

La fidelidad no diferencia: los cinco leen sin problema un texto corto y estructurado como el de una factura. La adherencia a JSON sí, porque una respuesta que no parsea corta la ejecución del flujo.

8. Asignación de modelo por tarea

Tarea	Modelo asignado	Justificación
Extracción de factura	Gemini 3.1 Flash-Lite	Volumen alto, tarea simple sobre texto limpio. Salida estructurada por esquema JSON, que elimina el modo de falla que importa. El más barato de los que garantizan el formato.
Reintento por confianza baja	Claude Haiku 4.5	Se aplica a un porcentaje mínimo de casos, así que el costo cuatro veces mayor es despreciable en el total. Al ser de otro proveedor, aporta una segunda lectura independiente en vez de repetir el mismo error.
Validaciones, cálculos, fechas	Ninguno	Determinístico y auditable. Un LLM sería más caro, más lento y menos confiable.
Mensajes y correos	Ninguno, plantillas	Texto fijo con variables. Ver punto 2.

Tareas previstas para la fase 2

El alcance de esta entrega se acotó deliberadamente. Estas tareas quedan documentadas porque cambian la asignación de modelos, y sirven para mostrar que la decisión no es "el barato siempre".

Tarea futura	Modelo asignado	Justificación
Cotejo de factura contra orden de compra	Claude Sonnet 5	Lectura densa: comparar dos documentos, detectar faltantes, diferencias de cantidad y precio. Es razonamiento sobre información dispersa, no extracción. Acá el modelo barato sí falla, y el costo mayor se justifica porque la tarea lo exige.
Deduplicación semántica de proveedores	Claude Haiku 4.5	Comparación de nombres y domicilios con variantes. Tarea simple pero con matices que una regla exacta no cubre.
Recarga histórica de facturas	Gemini 3.1 Flash-Lite vía Batch API	Volumen masivo sin urgencia. El procesamiento por lotes reduce la tarifa a la mitad.
Análisis de	Claude	Interpretación de datos comerciales con impuestos y comisiones. Razonamiento sobre números, no

márgenes por canal de venta	Sonnet 5	lectura.
-----------------------------	----------	----------

El patrón: **extracción y clasificación van a modelos económicos; comparación y razonamiento, a modelos medios; nada va a modelos de frontera**, porque ninguna tarea del sistema lo requiere.

9. Decisión

Modelo principal: Gemini 3.1 Flash-Lite. Modelo de reserva: Claude Haiku 4.5.

Los cinco modelos leen bien una factura: la tarea no los exige. Con la calidad empatada, decide el costo, y ahí quedan dos candidatos separados por centésimas. Entre esos dos, Flash-Lite se impone por su salida estructurada por esquema JSON, que elimina el único modo de falla que importa: una respuesta que no parsea y corta la ejecución.

Pagar Sonnet 5 o GPT-5.6 Sol para esta tarea sería gastar entre 11 y 24 veces más por una capacidad de razonamiento que el flujo no usa, porque las cuentas las hace un nodo de código.

El escalado a Haiku se dispara cuando el modelo principal devuelve confianza baja o una respuesta inválida. Antes de molestar a una persona, el sistema pide una segunda lectura a un modelo de otro proveedor. Si esa segunda lectura coincide, se continúa; si no, va a revisión humana.

10. Por qué el riesgo del modelo barato está acotado

La elección de un modelo económico no descansa en la confianza en el modelo, sino en que el diseño del flujo limita el daño de un error:

- **Las cuentas no las hace la IA.** Un nodo de código recalcula neto + IVA contra el total. Si la IA leyó mal un número, la verificación falla y la factura se deriva.
- **Hay un humano en el medio.** Toda discrepancia, confianza baja o monto sobre el tope va a aprobación por Telegram.
- **La entrada está saneada.** El modelo recibe texto limpio de un PDF digital validado, no una imagen ni un archivo arbitrario.
- **Hay una segunda opinión antes del humano.** El escalado a Haiku filtra los errores de lectura ocasionales sin intervención.

Un error del modelo no produce un pago incorrecto: produce una derivación a revisión. Ese es el argumento que justifica no pagar por un modelo de frontera.

11. Palancas de reducción aplicadas

Palanca	Ahorro	Estado
Tareas sin IA	100% en 6 de 8 tareas	Aplicada. Cálculos, validaciones y redacción resueltos con código y plantillas.
Validación en cascada previa	~30% de las llamadas	Aplicada. Siete de los nueve códigos de rechazo vigentes (RC-01 a RC-07) se resuelven antes de la IA y no consumen tokens.
Solo PDF con texto seleccionable	Evita el modelo con visión	Aplicada. Un modelo multimodal costaría varias veces más por la misma lectura.
Límite de tokens de salida	Acota el peor caso	Aplicada. Tope de 2.500 tokens en el nodo del modelo. Ver punto 12: en el entorno de prueba el tope no puede fijarse por el tamaño de la respuesta útil.
Caché de prompt	Hasta 90% sobre la entrada cacheada	Disponibile. El prompt de sistema es idéntico en cada llamada.
Procesamiento por lotes	50% sobre todo	Previsto para la recarga histórica. No aplica al flujo en vivo, donde las facturas entran de a una.

La cascada de validaciones es la palanca más relevante y ya está en el flujo: lo que está mal formado se rechaza antes de gastar un solo token.

12. Nota sobre el entorno de prueba

Las corridas de este trabajo se ejecutaron con modelos de tier gratuito de OpenRouter, por tratarse de un entorno académico sin presupuesto asignado.

Sobre OpenRouter. Es una pasarela: expone con una única API y una única credencial el catálogo de varios proveedores. El flujo no se integra contra un proveedor en particular, sino contra la pasarela, y el modelo es un parámetro de un campo. Esa indirección es lo que permitió sustituir el modelo siete veces sin tocar una línea del flujo.

12.1 Modelo en uso

Parámetro	Valor
Modelo	<code>inclusionai/ling-3.0-flash-fin:free</code>
Máximo de tokens de salida	2.500
Temperatura	0
Tokens de prompt medidos	~780

La temperatura en cero es deliberada: la tarea es leer números impresos, no redactar. No hay ninguna razón para querer variación entre corridas del mismo comprobante, y el determinismo hace el resultado auditable.

12.2 La rotación forzada de modelos

Siete modelos gratuitos quedaron descartados durante el desarrollo, por cuatro causas distintas:

Modelo	Causa del descarte
<code>google/gemma-4-31b-it:free</code>	Límite de peticiones persistente
<code>minimax/minimax-m3:free</code>	Retirado del catálogo gratuito
<code>nvidia/nemotron-3.5-lightning:free</code>	Modelo de razonamiento: agota el presupuesto deliberando
<code>openai/gpt-oss-20b:free</code>	Pasó a ser solo de pago
<code>dots-studio/dots-3-note-preview:free</code>	Devuelve cadena vacía ante un pedido de extracción
<code>deepseek/deepseek-v4-flash-0731:free</code>	Retirado del catálogo gratuito
<code>nex-agi/nex-n2.5-pro:free</code>	Retirado del catálogo gratuito
<code>openrouter/free</code> (router automático)	Elige modelos no aptos para la tarea; respuestas vacías

Cuatro de los ocho dejaron de estar disponibles en cuestión de días. **La conclusión no es sobre la calidad de los modelos sino sobre el tier gratuito: el catálogo cambia sin aviso y no admite planificación.** Es una limitación que no aparece en ninguna tabla de precios y que, en una implementación productiva, se elimina contratando un tier pago — cuyo costo, según el punto 5, es de centavos mensuales.

12.3 Hallazgo: el tope de tokens y los modelos de razonamiento

El dato más relevante de la implementación, y el que corrige la intuición sobre cómo fijar el límite de salida.

Con el tope en 600 tokens —un valor generoso frente a los ~250 que ocupa el JSON de respuesta— el nodo devolvió **cadena vacía**, sin error. La llamada figuraba como exitosa. El diagnóstico apareció en la metadata de la respuesta:

```
"generationInfo": { "finish_reason": "length" },
"tokenUsage": { "completionTokens": 600, "promptTokens": 780, "totalTokens": 1380 }
```

El modelo consumió los 600 tokens completos y no llegó a emitir el JSON. **Los tokens de razonamiento interno cuentan contra el tope de salida, pero no aparecen en el campo de texto de la respuesta.** El presupuesto se agota antes de que se escriba el primer carácter útil.

Consecuencias prácticas:

- **El tope no puede dimensionarse por el tamaño de la respuesta esperada.** Hay que sumarle el razonamiento interno, que es invisible y varía por modelo.
- **Un tope demasiado ajustado no ahorra: rompe.** Una respuesta truncada deriva la factura a revisión humana, que cuesta más que los tokens que se intentaba economizar.
- **El síntoma es silencioso.** No hay error, no hay excepción: hay una cadena vacía y una ejecución exitosa. Sin mirar `finish_reason` el diagnóstico es imposible.

El tope de 2.500 surge de ahí: cubre el razonamiento observado más el JSON, con margen. Sigue siendo un límite explícito que acota el peor caso —que es lo que la optimización de costos busca— pero fijado sobre una medición y no sobre el tamaño del entregable.

En una implementación productiva con el modelo elegido en la matriz, que no antepone cadena de razonamiento, el mismo tope sería varias veces mayor a lo necesario y podría bajarse sin riesgo.

12.4 Validez del análisis

La matriz de decisión se construyó sobre los **precios de lista publicados por cada proveedor**, que son los que aplicarían en una implementación productiva. Que el entorno de prueba use un modelo distinto no invalida el análisis: la tarea de extracción es la misma, y el diseño del flujo —cálculos en código, verificación aritmética, aprobación humana— es independiente del modelo que se enchufe. La rotación de siete modelos sin modificar el flujo es la prueba de esa independencia.

Precios verificados en septiembre de 2026. Las tarifas de API cambian con frecuencia: verificar contra la página oficial de cada proveedor antes de tomar decisiones de presupuesto.